Debugging in Wasmtime: Recording, Snapshots, and Reversibility

Chris Fallin

Bytecode Alliance Plumber’s Summit, Oct 2026

Guest debugging landed in ~March

- `wasmtime run -g1234 file.wasm`
- Attach LLDB and debug “like normal”
  - It pretty much works! Successful reports “from the field”

- … so we’re done?
  - **Debug failures in the field**: record/replay
  - **Tackle difficult heisenbugs**: *deterministic* record/replay
  - **Root-cause state corruption quickly**: watchpoints + reversibility
  - **Have a nice experience**: builtin DAP, DWARF parsing, TUI

Record/replay in a nutshell

What if you could:

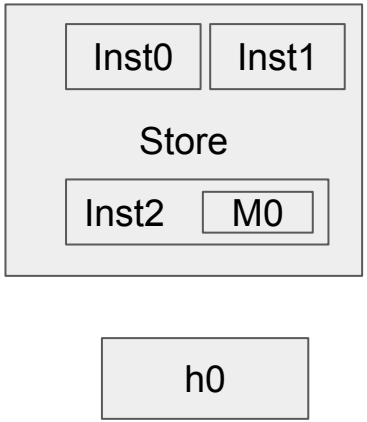
- Have an “always-on” flight data recorder in production (FaaS, dataplane, …),
- That saves off its trace buffer on any crash/bug (trap, error return code, …),
- And permits you to download and replay that trace locally?

We would need:

- Lightweight trace format (not full instructions or memory; only IO boundary);
- A stable format, at the level of e.g. the Wasm standard –
  we cannot tie SDK version and server-side Wasmtime version together;
- Low-level hooks in Wasmtime to record; and
- An alternate “driver loop” in Wasmtime to replay

Deterministic Wasm record/replay

Record



Action

```
Host
  - instantiate inst0, …
  - call inst0.f
    - write M0
    - call inst1.g
      - write M0
    - call inst2.h
       - call h0
         - write M0
```

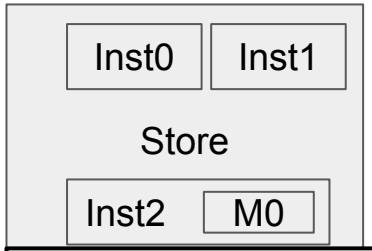
Trace

```
- instantiate …
- enter inst0.f
   - args: …
- host-call h0
- update M0[a..b]
- host-ret h0
   - rets: …
- exit inst0.f
```

Idea 1: record only *core Wasm entries/exits* and *memory updates*

Deterministic Wasm record/replay

Replay



Trace

```
- instantiate …
- enter inst0.f
   - args: …
- host-call h0
- update M0[a..b]
```

Action

```
Host (following trace)
  - instantiate inst0, …
  - call inst0.f
    - write M0
    - call inst1.g
```

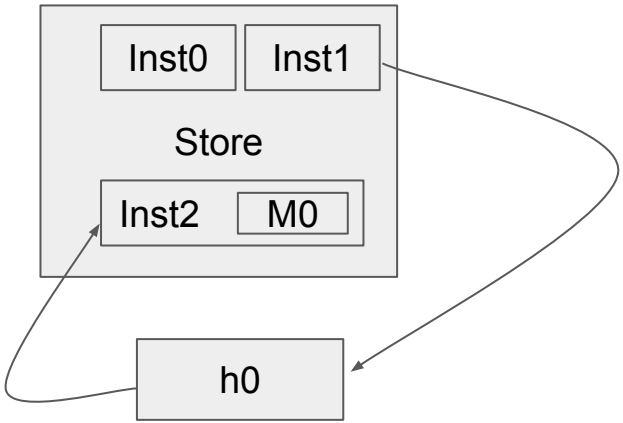
Idea 2: replay *takes over whole store* and replays host-API actions (instantiation, call)

Idea 3: replay *replaces hostcalls with stubs* that have the same effects (corollary: no actual host APIs are needed for replay; stock Wasmtime!)

Components?

- Yes! They're supported! All the way up to CM-async and multiple tasks!
- No! They don't appear in the trace *at all*
- The trace is a lower abstraction level: Wasm activations, entries, exits. That's it. Small clean abstraction is *necessary for later steps*.
- Also makes traces portable across Wasmtime versions. We only speak of core-Wasm spec-level entities: instances, functions, memories.
- Contra earlier assumptions: shared-nothing boundary is not *necessary* for deterministic recording; it just makes it much more efficient

Replay driver loop and reentrancy




```
Host
  - instantiate inst0, …
  - call inst0.f
    - call h0
      - call inst2.callback
```

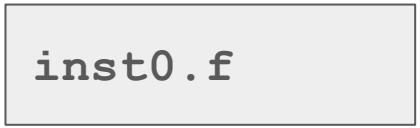
How do we replay this trace?

1. Stub for h0 continues to read trace; recursively invokes replay on inst2.callback; OR
2. Stub for h0 “suspends”, then toplevel invokes inst2.callback

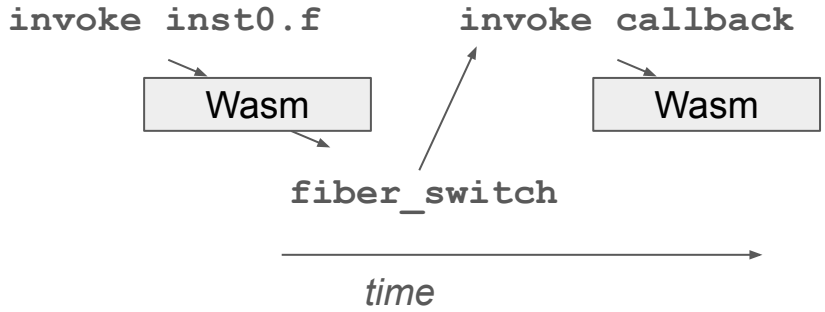
Replay driver loop and reentrancy

```
loop {
  match trace.read() {
    Instantiate => …,
    Enter => {
      new fiber; switch;
    }
    HostCall => {
      assert fiber suspended;
      inject return value;
      switch;
    } … } … }
```









Vs. Earlier Prototype

- Some of you may remember that we had a record/replay prototype in 2025!
  - Internship project by Arjun Ramesh (CMU) with Ben Titzer + me
- Arjun's prototype was great work! Showed low overhead + possibility

- A few things I wanted to change, leading to rewrite:
  - Realization that we *do not want to embed component-model details*
    - Complexity + invasiveness of implementation
    - Unnecessary limit on capabilities
  - Replay worked by rebuilding full host/Wasm sandwiches on stack; *deeply incompatible with snapshots + reversibility*

Snapshotting

- Goal: provide a primitive for reversible debugging
- In the replay API, provide a primitive to:
  - “Take a snapshot” (opaque handle, may hold resources behind-the-scenes)
  - “Rewind to snapshot” (implicitly invalidates later snapshots, GC handles, frame handles, …)
- This must work at *arbitrary* points where we have control
  - E.g., at a breakpoint, with many Wasm frames on the stack
  - … and a “host/Wasm sandwich” with lots of reentrancy
  - … and with lots of arbitrary Store state (e.g. GC roots) that may be time-dependent
    - *(but remember that during replay, we don’t have a normal T in Store<T>; WASI is “provided” by the trace’s hostcall answers)*
- How are we going to snapshot live stacks?!
  - **Carefully.**

Snapshotting: Execution Stacks

- During replay, we keep each Wasm activation on its on fiber, and yield back to a driver loop on any call “out”. That’s important!
- Observation: if
  - A fiber stack has *no Rust frames* (whose reexecution is UB), and
  - The executing code’s resources do not move (no-move memories/tables, fixed stack addr, fixed module addr) within one replay lifetime,
- Then It should be valid to **memcpy a snapshot of a stack, and restore it.**
  - *(Live pointers and all! Note that the top-of-stack is part of what we restore)*
- We can provide this as a primitive *in the fiber library*
- But: “no Rust frames”?!
  - Yes: new “raw fibers” primitive; special trampolines; direct call from Cranelift code to handwritten-assembly fiber switch on “outbound” calls

Snapshotting: everything else

- Oh, and we need to save/restore the whole `Store` too.
- So… `#[derive(Clone)]` and call it a day??
- No: need a strategy for each element of the store.
  - User T: not present (trace takes its place). Trace itself snapshots by just noting/rewinding the replay pointer.
  - Tables, globals: full copy. (Optimization if tables are never written.) Better answer TBD.
  - GC roots, heap and collector state: full copy.
    - Rewinding *also* rewinds GC state; an object can be “uncollected” if we go backward in time. “Very simple” if we think of GC as part of guest state fully, not host state.
  - Memories: *persistent immutable snapshots library* + *watchpoints*
    - Use the software watchpoint mechanism (PR open) to “write-protect” whole pages; on write, note page is dirty, take a copy.

Snapshotting: important simplifications

- We only support this in trace-replay mode.
  - (We can’t snapshot and rewind the outside world!)
- A snapshot is a live-in-this-replay object, not anything persistent.
  - (Allows punting on live pointer relocation, etc.)
- We only need a snapshotting strategy for any state that is still kept in replay.
  - (So WASI is exempt; so is arbitrary embedder code.)
- We will not support “forking” or execute-from-snapshot.
  - (Trace-replay mode is fundamentally read-only; supporting divergence in a best-effort way is a hard research problem.)
  - (This has consequences for debuggers: all debugger logic needs to be *in the host*, not in the debuggee Wasm.)

Reversible debugging

- Boothe, PLDI 2000: “Efficient Algorithms for Bidirectional Debugging”
- How do you reverse-continue to a breakpoint/watchpoint/next inst/…?
- Answer: two-pass algorithm
  - Keep snapshots throughout history (ideally: exponential spacing back from present)
  - Rewind to nearest snapshot; play forward once, and count the event
  - Then play forward again, an stop at N-1th
  - How do we do reverse-step? New kind of precise fuel

Does the ecosystem support reverse debugging?

Surprisingly, yes! … But it's awkward.

```
(lldb) process continue --reverse
```

And: I haven't tested it much. Definitely off the "well-paved path".

Part 2: ergonomics and batteries-included

- Using LLDB and gdbstub protocol has been great, but also can be frustrating for users
  - “What do you mean I can’t use my lldb I already have installed”
  - Awkward lack of errors if pieces are missing – we just miss breakpoints
  - A few unresolved bugs I never bottomed out
  - No exposure of native Wasm concepts: multimemory, tables, globals, GC refs and objects, the operand stack and locals, …
- Can we build a first-class Wasm debugging experience?

DAP: Debug Adapter Protocol

- Fundamentally a higher abstraction level: protocol works ~at the UI level, presenting an object hierarchy ~as the user sees it (so all DWARF parsing, data structure interpretation, and knowledge of source language is in the DAP server, not the UI)
- Let's write a DAP server that presents info from DWARF parsing but also natively shows a hierarchy of Wasm objects!
- … isn't it hard to parse DWARF? And pretty-print arbitrary C++ and Rust data structures? Don't you need a whole Python interpreter for that?
  - … this is the age of robots
  - … my implementation supports:
    - Rust: str, String, Vec, HashMap, HashSet, BTreeMap, BTreeSet, Box, Mutex, *Cell, Arc/Rc, ManuallyDrop, Pin, … (30-ish total)
    - C++: string, vector, map, set, unordered_map, unordered_set, smart pointers (libc++)
    - C: C strings
  - Full support for "access path" minilanguage: `p vec[0]`, `watch vec[0]`, `p &vec[0]`, …

Demo

Next Steps

- Land everything! RFC? + ~30kLoC of PRs (~25 planned)
  - Open question: split between what goes in Wasmtime and what goes into Dapper
    - We've intentionally created a "platform interface" – we don't have to be monolithic (but batteries-included might be nice)
- Further development and testing with DAP
- Dynamic languages
  - Original RFC's plan was to have a DAP multiplexer
  - I am not sure that this is compatible with record/replay + reversibility
    - Cannot have "live execution" of runtime bits implementing debugger within the Store
  - DWARF in NightMonkey + pretty-printer extensions in DWARF-DAP component?
    - We could have metadata attaching to the *interpreter* too; we just need to invent a format
- More field usage and feedback!